TUNA: Tuning Unstable and Noisy Cloud Applications

EuroSys 2025
Presenter: Rahul Saha

Motivation

- ML-based autotuners widely used (DBs, storage, clusters)
- Assume tuning environment ≈ deployment environment
- Cloud introduces:
  - Performance variability
  - Noisy neighbors
  - VM-level interference
- Result: unstable configs + slow convergence

How Modern Autotuners work

1 Optimizer Suggests Config → 2 Execute on System-Under-Test → 3 Measure Performance → 4 Update Optimizer

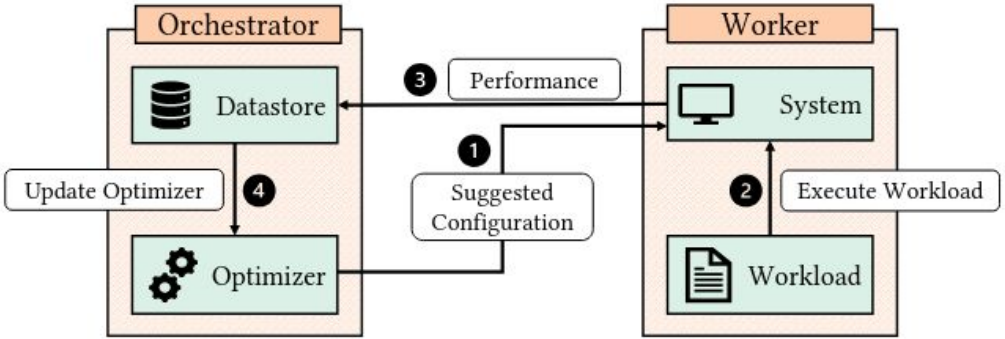


Figure 1. Modern auto-tuning system design [42, 98, 99]

What this Paper Investigates

- How does noise affect tuner convergence?
- How noisy is modern cloud infrastructure?
- Do tuners pick unstable configurations?
- Can we design a noise-aware tuner?

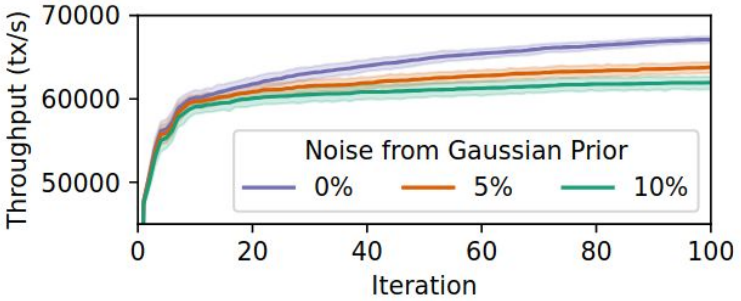
Even Small Noise Hurts Convergence

Experiment: Tune PostgreSQL on bare-metal (no real noise), then synthetically inject Gaussian noise at various levels.

2.5× slower convergence with just 5% noise

4.35× slower convergence with 10% noise

Figure 2. Optimizer Rate of Convergence for epinions workload, running on PostgreSQL 16.1 at various levels of noise.



A 68-Week Cloud Measurement Study

43,500+ VMs sampled

7M+ data points

68 weeks study duration

92 metrics tracked

Paper	Year	Duration	Samples	Instances	Platform	Disk	Memory	CPU	Network	OS
Schad et al. [75]	2010	4 weeks	6 k	4	A	✓	✓	✓	✓	✗
Iosup et al. [37]	2011	52 weeks	250 k	N/A[a]	A, G	✗	✗	✓	✗	✗
Farley et al. [25]	2012	2 weeks	59k	40	A	✓	✓	✓	✓	✗
Leitner and Cito [54]	2016	4 weeks	54k	82	A, M, G, I	✗	✓	✓	✗	✗
Maricq et al. [62]	2018	46 weeks	900 k	835	CL	✓	✓	✗	✓	✗
Figiela et al. [27]	2018	22 weeks	730 k	13,723[a]	A, M, G, I	✗	✗	✓	✗	✗
Scheuner and Leitner [76]	2018	4 weeks	63 k	244	A	✓	✓	✓	✓	✗
Uta et al. [87]	2020	3 weeks	1000 k	1	A, G, H	✗	✗	✗	✓	✗
De Sensi et al. [19]	2022	N/A	516 k	2	A, M, G, O	✗	✗	✗	✓	✓
This Work	**2024**	**68 weeks**	**7037 k**	**43,641**	M	✓	✓	✓	✗	✓

[a] These studies were conducted on serverless nodes

Table 1. Comparison of our work to prior benchmarking studies. For platform A indicates Amazon AWS, M indicates Microsoft Azure, G indicates Google GCP, I indicates IBM Cloud, O indicates Oracle OCI, CL indicates CloudLab, H indicates HPCCloud.

Platform Improvements Reduced Some Variability

- CPU CoV < 0.2%
- Disk CoV < 0.4%
- Much lower than older studies
- VM SKUs now tightly controlled

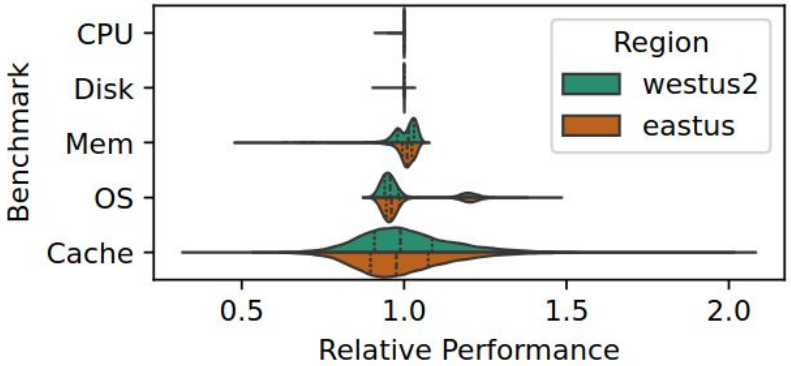


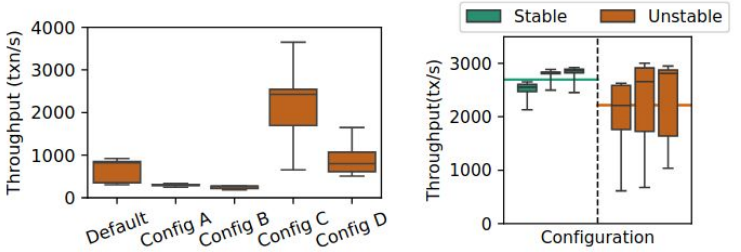
Figure 4. The variance of benchmarks targeting CPU, Disk, Memory, the OS, and CPU cache. Relative performance is relative to the mean performance seen. Higher is better.

Memory, Cache, and OS Still Noisy

- Memory CoV ≈ 4.9%
- Cache CoV ≈ 14.4%
- OS operations high variability
- VMEXIT and shared resources still problematic

Some Configurations Are Inherently Unstable

- Same config on different VMs → wildly different performance
- Up to:
  - 70% degradation
  - 36% CoV
- Root cause: query plan instability



(a) Throughput for the 5 configurations of the initialization set evaluated on the same 30 nodes.

(b) A subset of best-performing configs. when deployed on new nodes.

Figure 5. The performance of initialization and of best-performing configurations (picked by the optimizer) during training (left) and when deployed on new nodes (right).

Two Core Problems

- Noise slows optimizer convergence
- Optimizer selects unstable configs
- Single-node tuning is insufficient

TUNA overview

- Drop-in replacement for sampling strategy
- Does NOT modify:
  - Optimizer
  - System Under Stress
- Adds
  - Multi-fidelity sampling
  - Outlier detection
  - Noise modeling
  - Robust aggregation

$$y = f(config)$$

$$y = f(config) + \epsilon(machine, cloud\_noise)$$

TUNA

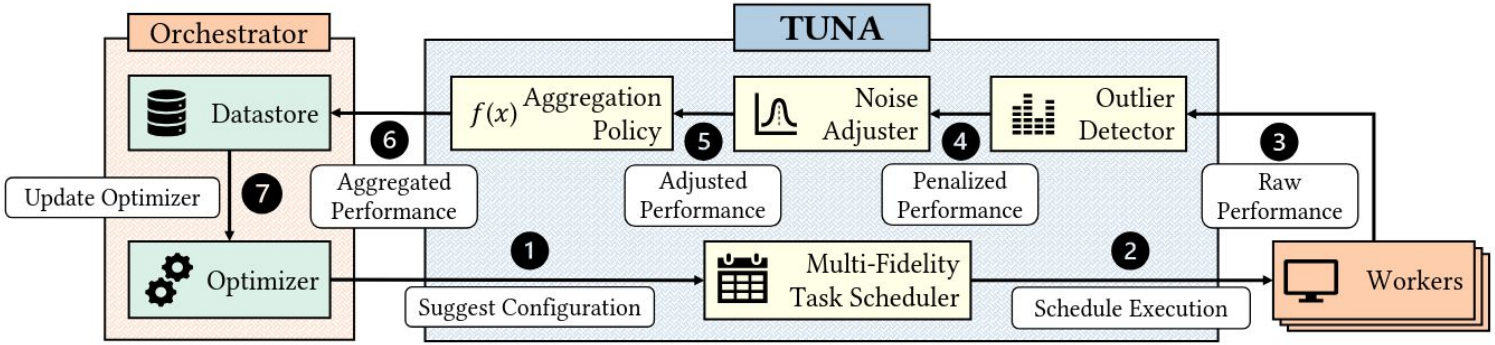


Figure 7. Design for TUNA with our contributions highlighted lime and blue.

Evaluate Promising Configs on Multiple Nodes

- Start with 1 node
- Promote promising configs to 3, then 10 nodes
- Use Successive Halving
- Benefits:
  - Faster filtering
  - Detect instability

Detecting Unstable Configs

- Relative Range
  - (max(x) - min(x))/ E(x)
- Threshold: 30%
- Conservative to avoid catastrophic deployment

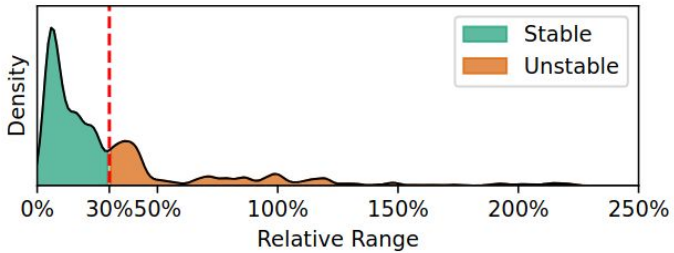
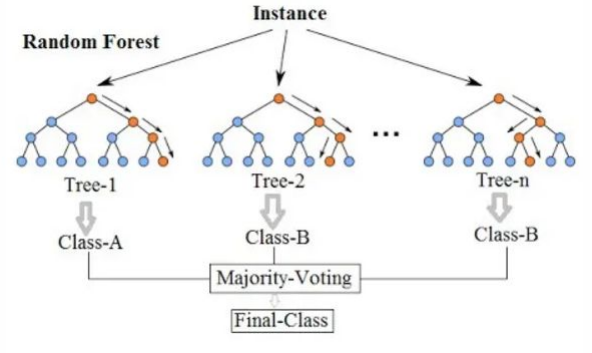


Figure 8. Sensitivity analysis of 1000 configurations seen during tuning. Using a red dashed line, we specify our detection threshold at 30% between the first and second peaks.

Predict the Noise-Free Performance Signal

- Use:
  - System metrics (CPU, memory)
  - Worker ID (one-hot)
- Model:
  - Random Forest
- Predict percent error from mean
- Retrain during tuning
- **Not** transfer learning - trained per run



Aggregation Policy: Optimize for Worst Case

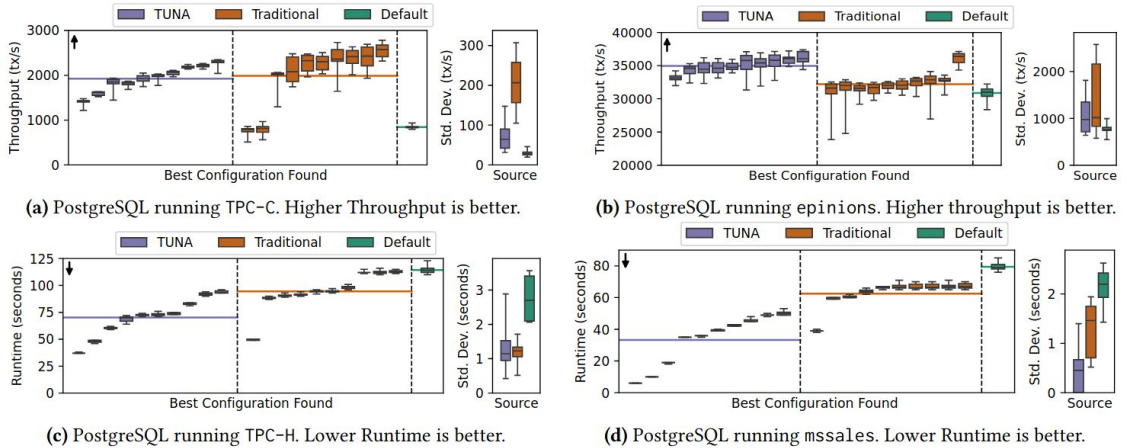
- Mean/median ignore outliers
- Optimizes worst-case performance
- Outlier detector bounds uncertainty

Evaluation Methodology

- 10-node cluster
- 8-hour tuning budget
- Compare:
  - TUNA
  - Traditional single-node sampling
  - Default config
- Deploy best config to 10 new nodes
- Systems
  - PostgreSQL
  - Redis
  - NGINX

PostgreSQL Across Workloads

- TPC-C:
  - Similar mean performance
  - Much lower variance
- epinions:
  - Higher throughput
  - Fewer unstable configs
- TPC-H:
  - 38% faster than default
- mssales
  - 2.39x speedup over default



(a) PostgreSQL running TPC-C. Higher Throughput is better.

(b) PostgreSQL running epinions. Higher throughput is better.

(c) PostgreSQL running TPC-H. Lower Runtime is better.

(d) PostgreSQL running mssales. Lower Runtime is better.

Figure 11. Results of applying tuned PostgreSQL configs with several workloads to new systems. The horizontal lines are mean throughput (or runtime). In all cases, TUNA either improves performance, reduces variability, or both.

Redis & NGINX

- Redis
  - TUNA avoids crashes
  - 86.8% lower std dev
- NGINX:
  - Lower tail latency
  - More stable

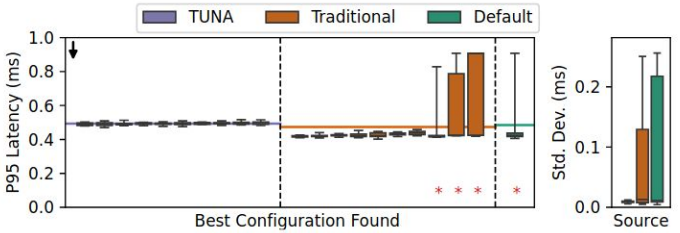


Figure 14. Redis running tuned YCSB-C workload configs. Lower is better. A red asterisk indicates that a node crashed.

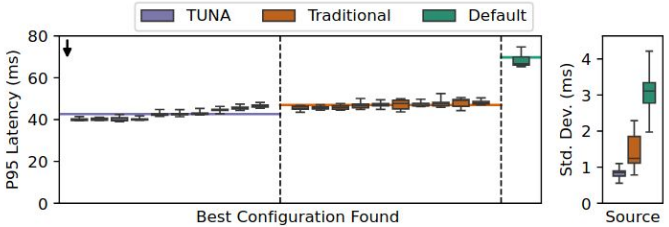


Figure 15. Results of applying tuned NGINX configs serving the top 500 Wikipedia pages. Lower is better.

Bare Metal & Region Robustness

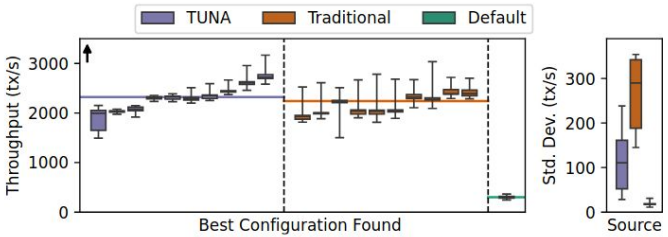


Figure 12. Results of applying tuned PostgreSQL configs running TPC-C in a new region. Higher throughput is better.

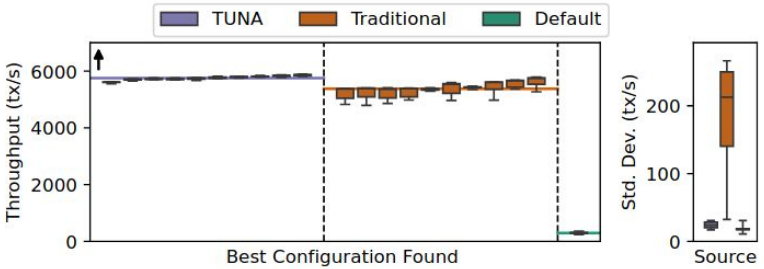


Figure 13. Results of tuned PostgreSQL configs running TPC-C on bare-metal CloudLab Nodes. Higher is better.

Takeaway

Autotuning must be cluster-aware — single-node tuning is fundamentally insufficient in modern cloud environments.